BULK CERTIFICATE GENERATOR

Complete Engineering Architecture, Implementation Lifecycle & Production Specification

Document Type: Full Lifecycle Engineering Specification	**Project Version:** 1.0.0 (Production-Ready)
Core Stack: Python 3.11+, FastAPI, SQLAlchemy 2.0, ReportLab	**Database:** SQLite WAL Mode / PostgreSQL Ready
Total Cost Model: $0.00 / ■0.00 (Zero-Cost Verified)	**Test Coverage:** 90% Statement Coverage (31 Tests Passing)
Author / Engineer: Backend Systems Engineering	**Generated Date:** October 10, 2026

Executive Summary & Core Engineering Observations

The Bulk Certificate Generator Backend API is a high-throughput, fault-isolated asynchronous system designed to ingest bulk batches of certificate recipients, render high-resolution vector PDF certificates, track granular job progress, and serve single and batched ZIP downloads under a strict **$0.00 / ■0.00 Zero-Cost Constraint**. Rather than relying on heavy headless browsers or expensive paid cloud APIs, the system uses a pure programmatic ReportLab vector engine and a database-backed worker architecture.

Key Engineering Discoveries & Observations

Observation 1: The Vector Engine vs. Headless Browser Paradigm
Headless Chromium (Puppeteer/Playwright) incurs ~300 MB RAM per browser process and 1,000–2,000ms per certificate, producing rasterized bitmap PDFs of 500 KB–2 MB. In contrast, ReportLab compiles vector PostScript primitives in **~25 milliseconds** with **~1 MB peak RAM**, producing **~3.23 KB PDFs**. This represents a 60x throughput gain and 150x storage density improvement, allowing over 309,000 certificates per gigabyte.

Observation 2: The PaaS Ephemeral Filesystem Trap
Free cloud PaaS hosts (Render, Koyeb, Railway) execute containers on ephemeral root filesystems. Any certificate persisted to local disk is permanently destroyed upon container reboot, sleep, or redeploy. Furthermore, Render freezes free web containers after 15 minutes of inactivity, terminating background threads. True zero-cost cloud requires either **Oracle Cloud Infrastructure (OCI) Always Free VM** (200 GB permanent block storage, 0 sleep timeouts) or decoupling storage to an S3-compatible zero-egress bucket (Cloudflare R2).

Observation 3: SQLite WAL Concurrency Mechanics
Default SQLite locks the entire database file during write transactions, causing 'database is locked' errors during concurrent API reads. Applying connection-level pragmas (PRAGMA journal_mode = WAL; and PRAGMA busy_timeout = 5000;) separates the write-ahead log from the database file, allowing continuous non-blocking reads while the worker writes batch updates.

Observation 4: Eliminating the Dual-Write Hazard
Using Celery with Redis creates a dual-write vulnerability: writing to SQL and pushing to Redis are separate operations that can desynchronize if one fails. The database-backed worker inserts parent GenerationJob and all child CertificateRecord rows in a single atomic ACID transaction, eliminating orphaned tasks.

Phase 1 & 2: Requirement Analysis & System Architecture

Functional Scope: The system ingests up to 1,000 recipients per bulk request, performs Pydantic validation, enforces uniqueness within the payload, records individual recipient outcomes, dynamically scales font sizes to prevent text overflow, generates cryptographic verification codes (V-XXXX-XXXX-XXXX), and packages completed certificates into streaming ZIP archives.

System Architecture: Structured as a Layered Modular Monolith with an asynchronous database-backed background worker:

Layer	Component	Responsibility
Presentation / API	FastAPI (ASGI) + Uvicorn	Request validation, JSON serialization, HTTP 202 Accepted, rate & payload limiting.
Persistence	SQLAlchemy 2.0 + SQLite WAL	ACID transaction management, multi-counter state machine, job querying.
Execution / Worker	Database-Backed Background Worker	Compare-and-Swap atomic claiming, per-recipient failure isolation, heartbeat tracking.
Rendering Engine	ReportLab Vector Compiler	A4 landscape vector PDF generation, dynamic font auto-scaling, pypdf integrity check.
Storage	Local FS / Persistent Named Volume	Secure storage hierarchy: ./storage/certificates/{job_id}/{cert_id}.pdf.

Comparison of Concurrency & Queue Strategies

Strategy	Crash Durability	Memory (RAM)	Infra Cost	Verdict
Synchronous HTTP	None (Lost if client disconnects)	Low	■0	Rejected: Gateway timeouts on >50 certs
FastAPI BackgroundTasks	Zero (Destroyed on server restart)	Low	■0	Rejected: Unsafe for durable production
Celery + Redis	High (Persisted in Redis queue)	High (>250 MB)	High (Needs Redis)	Rejected: OOM risk in 512MB free tiers
Database Worker (Chosen)	Full (Persisted in ACID SQL)	Very Low (<60 MB)	■0 (Zero Cost)	Optimal: Fault-tolerant, self-healing, ■0

Phase 3 & 4: Database Design & RESTful API Specification

The relational schema decouples parent job tracking from recipient certificate instances. Job progression is maintained via 5 synchronized counters: total_recipients, successful_count, failed_count, pending_count, and processing_count.

Table	Key Columns	Types & Constraints	Purpose
generation_jobs	id (PK) event_name issuer_name status total_recipients successful_count failed_count	VARCHAR(36) VARCHAR(255) VARCHAR(255) VARCHAR(50), Indexed INTEGER INTEGER INTEGER	Tracks batch lifecycle, status transitions (PENDING -> PROCESSING -> COMPLETED / COMPLETED_WITH_ERRORS), and metrics.
certificate_records	id (PK) job_id (FK) recipient_name recipient_email certificate_code verification_code status, file_path	VARCHAR(36) VARCHAR(36), Indexed VARCHAR(255) VARCHAR(255) VARCHAR(100), Unique VARCHAR(100) VARCHAR(50), VARCHAR(500)	Holds per-recipient outputs. Records granular errors (INVALID_RECIPIENT_DATA, RENDER_ERROR) on individual failures.

RESTful Endpoint Specifications

Method	Endpoint	Status	Description
POST	/api/v1/jobs	202 Accepted	Accepts bulk recipients, validates payload, inserts records, enqueues job.
GET	/api/v1/jobs/{job_id}	200 OK	Returns real-time multi-counter status, percentage, and download URL.
GET	/api/v1/jobs/{job_id}/certificates	200 OK	Lists recipient certificate rows with optional ?status= filtering.
GET	/api/v1/jobs/{job_id}/download	200 OK	Streams all successful PDFs packaged into a ZIP archive.
GET	/api/v1/certificates/{id}/download	200 OK	Downloads an individual certificate PDF file.
GET	/api/v1/health	200 OK	Service liveness and database connectivity health probe.

Phase 5 & 6: Vector Certificate Engine & Worker Processing

Certificate Rendering Architecture: ReportLab draws A4 landscape vector graphics (841.89 x 595.27 points). It features geometric borders, decorative corner florets, gold rosettes, dynamic text placement, and a cryptographic verification badge. To prevent long recipient names (e.g. 70 characters) from colliding with borders, the engine implements **Dynamic Font Auto-Scaling**:

```
# Dynamic Font Auto-Scaling Algorithm
max_width = page_width - 2 * (margin + 60)
font_size = 30
while font_size > 14:
    if canvas.stringWidth(recipient_name, 'Helvetica-Bold', font_size) <= max_width:
        break
    font_size -= 2
# Text is rendered at the dynamically calculated point size
```

Post-Generation Integrity Check: Every generated file is immediately opened with pypdf.PdfReader. If the page tree is corrupt or unreadable, the file is unlinked, and the record transitions to FAILED.

Worker Claiming & Concurrency Protection

To allow multiple worker processes to safely poll the same database without duplicate work, we use an **Atomic Compare-and-Swap SQL Update**:

```
claim_stmt = (
    update(GenerationJob)
    .where(GenerationJob.id == candidate_id, GenerationJob.status == 'PENDING')  # Guard condition
    .values(status='PROCESSING', started_at=now, updated_at=now)
)
result = db.execute(claim_stmt)
if result.rowcount == 0:
    return None  # Another worker claimed the job; back off safely
```

Per-Recipient Fault Isolation & Stale Recovery

The worker processes recipients in a loop where each recipient is wrapped in its own try...except block. If recipient #42 encounters a failure, its record is marked FAILED, the parent's failed_count is incremented, and the worker seamlessly proceeds to recipient #43. The batch concludes as COMPLETED_WITH_ERRORS.

Crash Recovery: If the worker process is terminated mid-batch, recover_stale_jobs() identifies jobs stuck in PROCESSING whose heartbeat (updated_at) is older than 300 seconds, resets in-flight records to PENDING, and preserves already generated certificates, enabling seamless resumption.

Phase 7 & 8: Defensive Security, Error Handling & Packaging

- **OWASP Security Headers:** Enforces X-Content-Type-Options: nosniff, X-Frame-Options: DENY, and X-XSS-Protection: 1; mode=block.
- **10 MB Payload Guard:** Custom middleware inspects Content-Length before reading the body, returning HTTP 413 Content Too Large on oversized payloads.
- **Path Traversal Shield:** Validates certificate IDs against ^[a-zA-Z0-9_-]+$ and verifies canonical paths via Path.resolve().relative_to(base_dir).
- **Non-Root Container:** Production Dockerfile executes under a dedicated non-privileged user (appuser, UID 1001).

Phase 10 & 11: Automated Testing & Empirical Benchmarks

Automated Test Suite: Built with Pytest, using isolated SQLite database fixtures, mock storage folders, and FastAPI's TestClient. Testing achieved **90% statement coverage** across 31 tests in 5.34 seconds:

Test Module	Tests	Key Behaviors Verified
test_validation.py	5 passed	Missing fields, empty recipient lists, whitespace trimming, duplicate emails.
test_pdf_generation.py	7 passed	Vector layout, long name font scaling, verification codes, pypdf integrity.
test_job_lifecycle.py	1 passed	End-to-end flow: Ingestion -> Enqueued -> Completed -> ZIP download.
test_failure_isolation.py	1 passed	Fault containment: corrupt recipient produces COMPLETED_WITH_ERRORS.
test_worker_recovery.py	2 passed	Worker crash detection, stale job reset, idempotent re-execution.
test_retrieval.py	3 passed	Single PDF retrieval, batch ZIP streaming, non-existent 404 responses.
test_security.py	6 passed	Path traversal attempts, incomplete download blocks, 10MB payload limits.
test_database_persistence.py	3 passed	SQLite WAL persistence across engine restarts and multi-counter updates.
test_health.py	3 passed	Liveness probes, database connection checks, readiness state.

Empirical Performance Benchmarks

Measurements captured via scripts/benchmark_performance.py across varying batch sizes:

Batch Size	Final Status	Total Time	Avg Time / Cert	Throughput	Peak RAM	Storage / PDF
N = 10	COMPLETED	0.286 s	28.64 ms	34.9 certs/s	1.08 MB	3.23 KB
N = 50	COMPLETED	1.289 s	25.78 ms	38.8 certs/s	0.82 MB	3.23 KB
N = 100	COMPLETED	2.490 s	24.90 ms	40.2 certs/s	0.94 MB	3.23 KB
N = 250	COMPLETED_WITH_ERRORS	6.332 s	25.33 ms	39.5 certs/s	1.34 MB	3.23 KB

Phase 12 & 13: Deployment, Backups & Production Readiness

Deployment Strategies:
• **Path A: Local Docker Compose:** Canonical multi-container setup (`cert_generator_api` and `cert_generator_worker`) sharing a named persistent Docker volume (`cert_data`).
• **Path B: Verified Cloud Tier (Oracle Cloud Always Free):** 2 AMD Compute VMs or 4 ARM Ampere cores with **200 GB permanent block storage at $0 forever**. Does not sleep after 15 minutes, avoiding the PaaS ephemeral storage trap.

Operations & Maintenance:
• **Online SQLite WAL Backup:** python scripts/backup_restore.py backup ./backups (safely backs up active database without blocking writers).
• **72-Hour Storage Retention Pruning:** python scripts/cleanup_retention.py (automatically removes job directories older than 72 hours).

Phase 14: Senior Engineering Interview Defense

Concise technical answers for architectural and systems design interviews:

Q1: Why choose a database-backed worker over Celery + Redis?

Eliminates the dual-write hazard between SQL and Redis by using a single atomic ACID transaction. Reduces infrastructure overhead and memory consumption to <60 MB, avoiding kernel OOM kills in free cloud containers.

Q2: Why not use FastAPI's built-in BackgroundTasks?

BackgroundTasks runs in process heap memory. If the server crashes, restarts, or redeploys, all queued work is permanently lost. A database-backed worker guarantees full disk persistence before returning HTTP 202 Accepted.

Q3: How do you prevent race conditions across multiple workers?

Using an Atomic Conditional SQL Update: UPDATE generation_jobs SET status='PROCESSING' WHERE id=:id AND status='PENDING'. Relational database engines serialize single-row updates. Exactly one worker receives rowcount == 1, while others receive 0 and back off.

Q4: How does SQLite support concurrency without locking up?

By configuring Write-Ahead Logging (PRAGMA journal_mode = WAL;) and a 5,000ms busy timeout. Readers query the database without blocking writers, and writers append to the -wal log without blocking readers.

Q5: What happens when 1 recipient out of 500 fails?

The worker isolates each recipient in an individual try/except block. Failed records are marked FAILED with an error code, the failed_count is incremented, and the worker continues. The job completes as COMPLETED_WITH_ERRORS.

Q6: How does the system recover from unexpected worker crashes?

The worker records an updated_at heartbeat on each certificate. On polling, recover_stale_jobs() resets jobs stuck in PROCESSING older than 300 seconds to PENDING. In-flight records revert to PENDING, while completed certificates remain intact.

Q7: Why ReportLab instead of Puppeteer / Headless Chrome?

ReportLab compiles vector PostScript graphics in ~25ms using ~1 MB RAM, producing crisp ~3.23 KB PDFs. Puppeteer uses ~300 MB RAM, takes ~1,500ms, and produces large 1 MB rasterized files.

Q8: How did you solve long recipient names overflowing certificates?

Dynamic Font Auto-Scaling: The engine measures name width in points using canvas.stringWidth() and iteratively decrements the font size from 30pt down to 14pt until it fits comfortably within layout boundaries.

Conclusion: The Bulk Certificate Generator demonstrates that production-grade durability, high throughput, and security do not require expensive cloud infrastructure. Through careful systems architecture, programmatic vector compilation, and transactional SQL state machines, enterprise-level performance can be achieved at $0 cost.